

Relatório final comparativo de branches

Repositório: GT-IoTEdu/sbr26_fingerprint

Objetivo

Este documento registra, de forma objetiva, as diferenças constatadas entre as branches **main**, **gpt_anna**, **claudio_anna** e **claudio_diego**. O foco está nas mudanças de estrutura, organização de código, pontos de entrada, funcionamento documentado e características técnicas observáveis.

Escopo analisado

- Organizacao geral do projeto e dos arquivos principais.
- Localizacao do pipeline de fingerprint.
- Localizacao da canonizacao e do hash.
- Organizacao da descoberta SSDP/UPnP/Nmap/ARP.
- Mudancas documentadas na forma de execucao.
- Documentacao adicionada ou atualizada em cada branch.

1. Visão geral das branches

Branch	Característica constatada
main	Estrutura original baseada nos scripts principais na raiz do projeto.
gpt_anna	Reorganização em pacote interno <code>iot_fingerprint/</code> , mantendo scripts antigos como wrappers compatíveis.
claudio_anna	Reorganização parcial em pacote interno <code>iotid/</code> , mantendo canonização e hash como arquivos da raiz.
claudio_diego	Extração da descoberta UPnP para <code>upnp_discovery.py</code> , mantendo o pipeline principal em <code>iot_id_fingerprint.py</code> e adicionando documentação específica.

2. Branch main

A branch main corresponde a estrutura original documentada do projeto.

- Mantém os quatro scripts principais diretamente na raiz: `iot_id_fingerprint.py`, `iot_net_scanner.py`, `canonicalize_features.py` e `fingerprint_hash.py`.
- O pipeline documentado envolve varredura ativa com `nmap`, captura com `dumpcap`, sonda TCP SYN com `nping`, extração passiva com `p0f`, extração com `tshark`, canonização e hash SHA-256.
- Não há pacote interno próprio para separar as responsabilidades do pipeline.

- A lógica fica organizada principalmente em scripts de raiz.

```
sbr26_fingerprint/  
■■■ iot_id_fingerprint.py  
■■■ iot_net_scanner.py  
■■■ canonicalize_features.py  
■■■ fingerprint_hash.py  
■■■ docs/  
■■■ testes/  
■■■ README.md
```

3. Branch gpt_anna

A branch gpt_anna reorganiza o projeto com uma camada de compatibilidade na raiz e um pacote interno de implementação.

- Cria o pacote iot_fingerprint/ para concentrar a implementação real.
- Mantém os scripts antigos na raiz como wrappers compatíveis.
- Move a canonização para iot_fingerprint/canonical.py.
- Move o hash para iot_fingerprint/hashing.py.
- Move a orquestração do pipeline para iot_fingerprint/pipeline.py.
- Centraliza SSDP, UPnP, Nmap UPnP e ARP em iot_fingerprint/upnp.py.
- Separa parsing p0f, extracao tshark, evidencias mobile e utilitários de runtime em módulos específicos.
- A forma de execução documentada permanece baseada nos scripts antigos da raiz.

```
sbr26_fingerprint/  
■■■ iot_id_fingerprint.py # wrapper compativel  
■■■ iot_net_scanner.py # wrapper compativel  
■■■ canonicalize_features.py # wrapper compativel  
■■■ fingerprint_hash.py # wrapper compativel  
■■■ iot_fingerprint/  
■■■ canonical.py  
■■■ hashing.py  
■■■ pipeline.py  
■■■ upnp.py  
■■■ p0f.py  
■■■ tshark_features.py  
■■■ mobile.py  
■■■ runtime.py
```

4. Branch claude_anna

A branch claude_anna também modulariza o projeto, mas com uma divisão diferente e mais próxima dos arquivos originais.

- Cria o pacote iotid/ para concentrar partes do pipeline.
- O arquivo iot_id_fingerprint.py passa a atuar como entrada CLI e despachante dos modos target e network.
- Move subprocessos, escrita de artefatos, logging, caminhos WSL, UPnP, p0f, tshark, descoberta e pipeline para modulos internos.
- Mantém canonicalize_features.py como arquivo principal da raiz.
- Mantém fingerprint_hash.py como arquivo principal da raiz.

- A canonização e o hash não são movidos para módulos internos equivalentes a canonical.py e hashing.py.
- A branch preserva uma estrutura em que os scripts da raiz continuam tendo papel mais explícito no projeto.

```
sbrc26_fingerprint/
■■■ iot_id_fingerprint.py
■■■ canonicalize_features.py
■■■ fingerprint_hash.py
■■■ iotid/
■■■ proc.py
■■■ artifacts.py
■■■ upnp.py
■■■ p0f.py
■■■ tshark.py
■■■ discovery.py
■■■ pipeline.py
```

5. Branch claude_diego

A branch claude_diego não cria um pacote interno amplo. A principal extração identificada e a centralização da descoberta UPnP.

- Adiciona upnp_discovery.py como arquivo compartilhado para SSDP/UPnP.
- Mantém iot_id_fingerprint.py como orquestrador principal do pipeline e CLI.
- Mantém canonicalize_features.py e fingerprint_hash.py como arquivos separados da raiz.
- Documenta upnp_discovery.py como ponto unico para ssdp_probe, fetch_upnp_description, nmap_upnp_scan e collect_upnp_identity.
- Adiciona docs/ARCHITECTURE.md com desenho do pipeline e fórmula do hash.
- Adiciona docs/USAGE.md com exemplos de uso.
- Documenta autodetecção de interface quando --iface e omitido.
- Documenta a opção --probe-ports para configurar as portas usadas na sonda SYN.

```
sbrc26_fingerprint/
■■■ iot_id_fingerprint.py
■■■ iot_net_scanner.py
■■■ canonicalize_features.py
■■■ fingerprint_hash.py
■■■ upnp_discovery.py

■■■ docs/
■■■ ARCHITECTURE.md
■■■ USAGE.md
```

6. Comparação sintética

Aspecto	main	gpt_anna	claude_anna	claude_diego
Estrutura principal	Scripts na raiz	Scripts wrappers + pacote iot_fingerprint/	Scripts + pacote iotid/	Scripts + upnp_discovery.py
Pipeline	Em iot_id_fingerprint.py	Em iot_fingerprint/pipeline.py	Em iotid/pipeline.py	Em iot_id_fingerprint.py
Canonizacao	canonicalize_features.py	iot_fingerprint/canonical.py via wrapper	canonicalize_features.py	canonicalize_features.py

Hash	fingerprint_hash.py	iot_fingerprint/hashin g.py via wrapper	fingerprint_hash.py	fingerprint_hash.py
UPnP/SSDP	Nos scripts originais	iot_fingerprint/upnp.py	iotid/upnp.py + iotid/discovery.py	upnp_discovery.py
Scripts antigos	Scripts com logica principal	Wrappers compatíveis	Entradas CLI/despacho	Scripts principais preservados
Documentacao	README	README atualizado	README atualizado	README + ARCHITECTURE + USAGE
Interface documentada	--iface informado	Uso documentado com scripts antigos	--iface no CLI	--iface pode ser autodetecta do
Portas da sonda	Portas fixas no exemplo	Portas fixas no pipeline	Portas fixas no pipeline	--probe-ports documentado

7. Diferenças de funcionamento documentadas

- Na main, a execução documentada exige informar diretório de saída, IP, tempo de captura e interface.
- Na gpt_anna, os comandos documentados permanecem baseados nos scripts antigos, mas esses scripts chamam a implementação interna em iot_fingerprint/.
- Na claudie_anna, iot_id_fingerprint.py trata os modos target e network e delega o processamento ao pacote iotid/.
- Na claudie_diego, a documentação registra autodetecção de interface quando --iface é omitido e adiciona --probe-ports para configurar as portas da sonda SYN.

8. Conclusão

- main: estrutura original baseada em scripts de raiz.
- gpt_anna: reorganização em pacote interno iot_fingerprint/, com scripts antigos mantidos como wrappers.
- claudie_anna: reorganização parcial em pacote interno iotid/, mantendo canonização e hash na raiz.
- claudie_diego: extração da descoberta UPnP para upnp_discovery.py, manutenção do pipeline principal em iot_id_fingerprint.py e adição de documentação técnica específica.

Referências consultadas

- Página de branches do repositório GT-IoTEdu/sbr26_fingerprint.
- README.md das branches main, gpt_anna, claudie_anna e claudie_diego.
- Arquivos principais das branches analisadas, incluindo iot_id_fingerprint.py, módulos internos e documentação técnica.